

```
#include <iostream>

#include <iomanip>

#include <string>

#include <ctime>

#include <fstream> // For file operations

using namespace std;

const int MAX_ACCOUNTS = 5;

const int MAX_HISTORY = 100;

struct Transaction {

    string type;

    double amount;

    string timestamp;

    Transaction* next;

};

struct Account {

    int accountNumber;

    int pin;

    double balance;

    Transaction* historyHead;

};

Account accounts[MAX_ACCOUNTS] = {

    {1001, 1234, 5000.0, nullptr},

    {1002, 2345, 10000.0, nullptr},

    {1003, 3456, 7500.0, nullptr},

    {1004, 4567, 2000.0, nullptr},

    {1005, 5678, 3000.0, nullptr}

};
```

```

string getTimestamp() {
    time_t now = time(0);
    return string(ctime(&now));
}

// File logging function
void logToFile(const Account& acc, const string& action, double amount = 0.0) {
    ofstream file("transaction_log.txt", ios::app);
    if (file.is_open()) {
        file << "Account #" << acc.accountNumber
            << " | " << action;
        if (amount > 0.0) {
            file << " | Amount: $" << fixed << setprecision(2) << amount;
        }
        file << " | Time: " << getTimestamp();
        file.close();
    }
}

void addTransaction(Account &acc, string type, double amount) {
    Transaction* t = new Transaction{type, amount, getTimestamp(), acc.historyHead};
    acc.historyHead = t;
}

int authenticate(int accNo, int pin) {
    for (int i = 0; i < MAX_ACCOUNTS; i++) {
        if (accounts[i].accountNumber == accNo && accounts[i].pin == pin) {
            return i;
        }
    }
    return -1;}

```

```

void showHistory(Transaction* head) {
    int count = 0;
    cout << "\n--- Transaction History (Last 3-5) ---\n";
    while (head != nullptr && count < 5) {
        cout << head->type << " of $" << fixed << setprecision(2) << head->amount
            << " on " << head->timestamp;

        head = head->next;
        count++;
    }
    if (count == 0) {
        cout << "No transactions found.\n";
    }
}

void searchTransaction(Transaction* head, double amount) {
    bool found = false;
    while (head != nullptr) {
        if (head->amount == amount) {
            cout << head->type << " of $" << fixed << setprecision(2) << head->amount
                << " on " << head->timestamp;

            found = true;
        }
        head = head->next;
    }
    if (!found) {
        cout << "No transaction found with amount $" << fixed << setprecision(2) << amount << "\n";
    }
}

```

```

void sortTransactions(Transaction* head) {
    if (head == nullptr) {
        cout << "No transactions to sort.\n";
        return;
    }
    Transaction* transArr[MAX_HISTORY];

    int count = 0;
    while (head != nullptr && count < MAX_HISTORY) {
        transArr[count++] = head;
        head = head->next;
    }

    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (transArr[j]->amount > transArr[j + 1]->amount) {
                swap(transArr[j], transArr[j + 1]);
            }
        }
    }

    cout << "\n--- Transactions Sorted by Amount ---\n";
    for (int i = 0; i < count; i++) {
        cout << transArr[i]->type << " of $" << fixed << setprecision(2) << transArr[i]->amount
            << " on " << transArr[i]->timestamp;
    }
}

void fundTransfer(Account &sender) {
    int choice;

    cout << "\n--- Fund Transfer Options ---\n";

    cout << "1. Transfer to Same Bank Account\n";

```

```

cout << "2. Transfer to Other Bank\n";
cout << "Choose option: ";
cin >> choice;
int receiverAccNo;
double amount;
switch (choice) {
    case 1: {
        cout << "Enter receiver's account number: ";
        cin >> receiverAccNo;
        if (receiverAccNo == sender.accountNumber) {
            cout << "Cannot transfer to the same account.\n";
            return;
        }
        for (int i = 0; i < MAX_ACCOUNTS; i++) {
            if (accounts[i].accountNumber == receiverAccNo) {
                cout << "Enter amount to transfer: $";
                cin >> amount;
                if (amount <= 0 || amount > sender.balance) {
                    cout << "Invalid amount or insufficient balance.\n";
                    return;
                }
                sender.balance -= amount;
                accounts[i].balance += amount;
                addTransaction(sender, "Same Bank Transfer to #" + to_string(receiverAccNo), amount);
                addTransaction(accounts[i], "Same Bank Received from #" +
to_string(sender.accountNumber), amount);
                logToFile(sender, "Same Bank Transfer to #" + to_string(receiverAccNo), amount);
            }
        }
    }
}

```

```

        logToFile(accounts[i], "Same Bank Received from #" + to_string(sender.accountNumber),
amount);

        cout << "Successfully transferred $" << fixed << setprecision(2) << amount
        << " to account #" << receiverAccNo << "\n";

        return;
    }
}

cout << "Recipient account not found in same bank.\n";

break;
}

case 2: {
    cout << "Enter external bank account number: ";

    cin >> receiverAccNo;

    cout << "Enter amount to transfer: $";

    cin >> amount;

    double fee = 5.0;

    double totalAmount = amount + fee;

    if (amount <= 0 || totalAmount > sender.balance) {
        cout << "Invalid amount or insufficient balance (including fee of $5).\n";

        return;
    }

    sender.balance -= totalAmount;

    addTransaction(sender, "Other Bank Transfer to #" + to_string(receiverAccNo), amount);

    addTransaction(sender, "Transfer Fee", fee);

    logToFile(sender, "Other Bank Transfer to #" + to_string(receiverAccNo), amount);

    logToFile(sender, "Transfer Fee", fee);

    cout << "Transferred $" << amount << " to external account #" << receiverAccNo
        << " with fee $" << fee << ". Total deducted: $" << totalAmount << "\n";
}

```

```

        break;
    }
    default:
        cout << "Invalid option.\n";
    }
}

void atmMenu(Account &acc) {
    int choice;
    do {
        cout << "\n--- ATM MENU ---\n";
        cout << "1. Deposit\n";
        cout << "2. Withdrawal\n";
        cout << "3. Check Balance\n";
        cout << "4. Transaction History (3-5)\n";
        cout << "5. Search Transaction by Amount\n";
        cout << "6. Sort Transaction by Amount\n";
        cout << "7. Fund Transfer\n";
        cout << "8. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: {
                double amount;
                cout << "Enter amount to deposit: $";
                cin >> amount;
                if (amount > 0) {
                    acc.balance += amount;
                    addTransaction(acc, "Deposit", amount);
                }
            }
        }
    } while (choice != 8);
}

```

```

        logToFile(acc, "Deposit", amount);

        cout << "Deposit successful.\n";
    } else {
        cout << "Invalid amount.\n";
    }

    break;
}

case 2: {
    double amount;

    cout << "Enter amount to withdraw: $";

    cin >> amount;

    if (amount > 0 && acc.balance >= amount) {
        acc.balance -= amount;

        addTransaction(acc, "Withdrawal", amount);

        logToFile(acc, "Withdrawal", amount);

        cout << "Withdrawal successful.\n";
    } else {
        cout << "Insufficient funds or invalid amount.\n";
    }

    break;
}

case 3:

    cout << "Your current balance is: $" << fixed << setprecision(2) << acc.balance << "\n";

    logToFile(acc, "Check Balance");

    break;

case 4:

    showHistory(acc.historyHead);

    logToFile(acc, "Viewed Transaction History");

```



```

        break;
    case 5: {
        double amount;

        cout << "Enter amount to search: $";

        cin >> amount;

        searchTransaction(acc.historyHead, amount);

        logToFile(acc, "Search Transaction by Amount", amount);

        break;
    }
    case 6:
        sortTransactions(acc.historyHead);

        logToFile(acc, "Sorted Transactions by Amount");

        break;
    case 7:
        fundTransfer(acc);

        break;
    case 8:
        cout << "Exiting from ATM menu...\n";

        break;
    default:
        cout << "Invalid choice. Please try again.\n";

    }

} while (choice != 8);

}

int main() {

    int mainChoice;

    cout << "==== Welcome to the ATM Transaction System =====\n";

```

```
do {  
  
    cout << "\n1. Enter PIN\n2. Exit\nChoose an option: ";  
  
    cin >> mainChoice;  
  
    if (mainChoice == 1) {  
  
        int accNo, pin;  
  
        cout << "Enter your Account Number: ";  
  
        cin >> accNo;  
  
        cout << "Enter your PIN: ";  
  
        cin >> pin;  
  
        int index = authenticate(accNo, pin);  
  
        if (index != -1) {  
  
            cout << "\nLogin successful! Welcome, Account #" << accounts[index].accountNumber << "\n";  
  
            atmMenu(accounts[index]);  
  
        } else {  
  
            cout << "Incorrect account number or PIN.\n";  
  
        }  
  
    } else if (mainChoice != 2) {  
  
        cout << "Invalid option. Try again.\n";  
  
    }  
  
} while (mainChoice != 2);  
  
cout << "\nThank you for using the ATM system. Goodbye!\n";  
  
return 0;  
}
```